

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Lab Title:

Input/Output, Data Types, Operators and Conditional Statements

Objectives:

- Learn input/output in Python
- Understand data types
- Use operators
- Apply if/else conditions

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)

- internet connection (optional for installation and help)
- basic knowledge of programming concepts

Theory:

Python uses:

- `input()` → take user input
- `print()` → display output
- Data types: int, float, string
- Operators: +, -, *, /
- Conditional statements: if, else

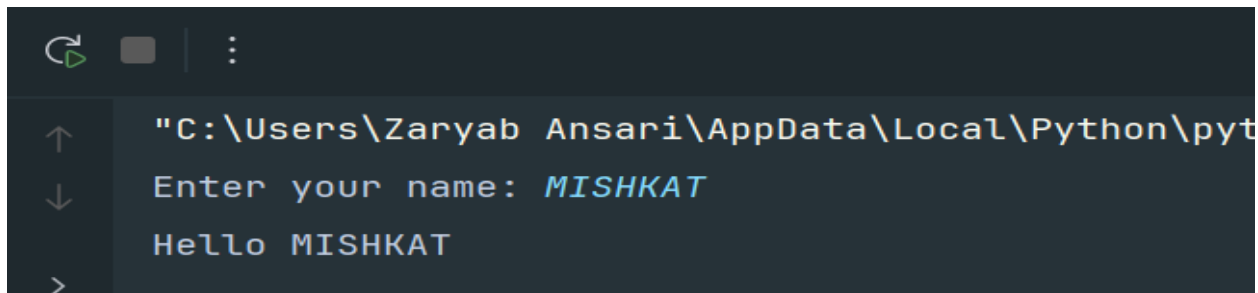
LAB WORK:

TASK 1:

```
name = input("Enter your name: ")
```

```
print("Hello", name)
```

OUTPUT:



```
C:\Users\Zaryab Ansari\AppData\Local\Python\pyt
Enter your name: MISHKAT
Hello MISHKAT
```

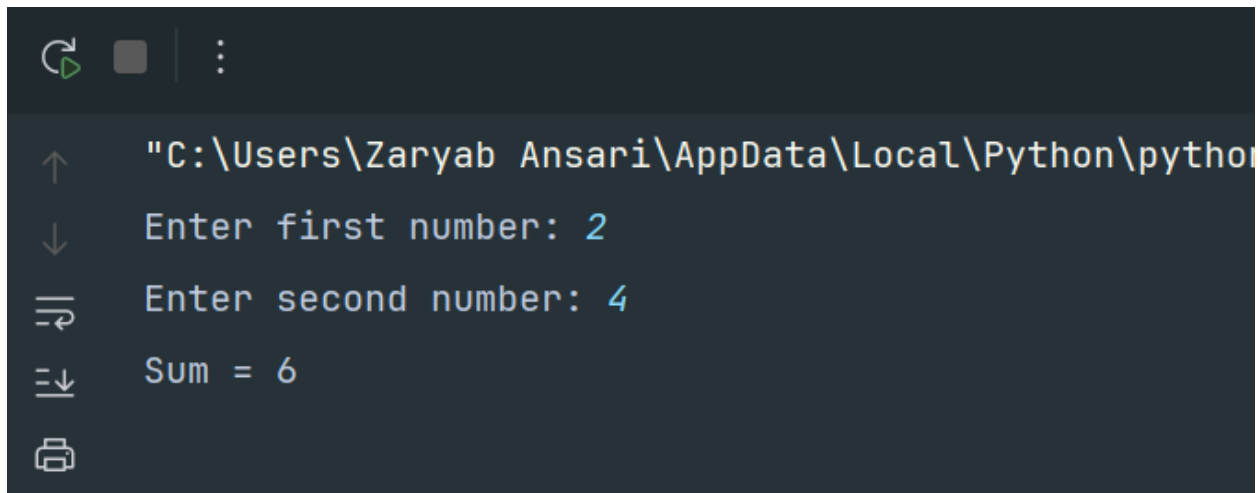
TASK2:

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

```
print("Sum =", a + b)
```

OUTPUT:



```

C:\Users\Zaryab Ansari\AppData\Local\Python\python
Enter first number: 2
Enter second number: 4
Sum = 6

```

TASK 3:

```
num = int(input("Enter a number: "))
```

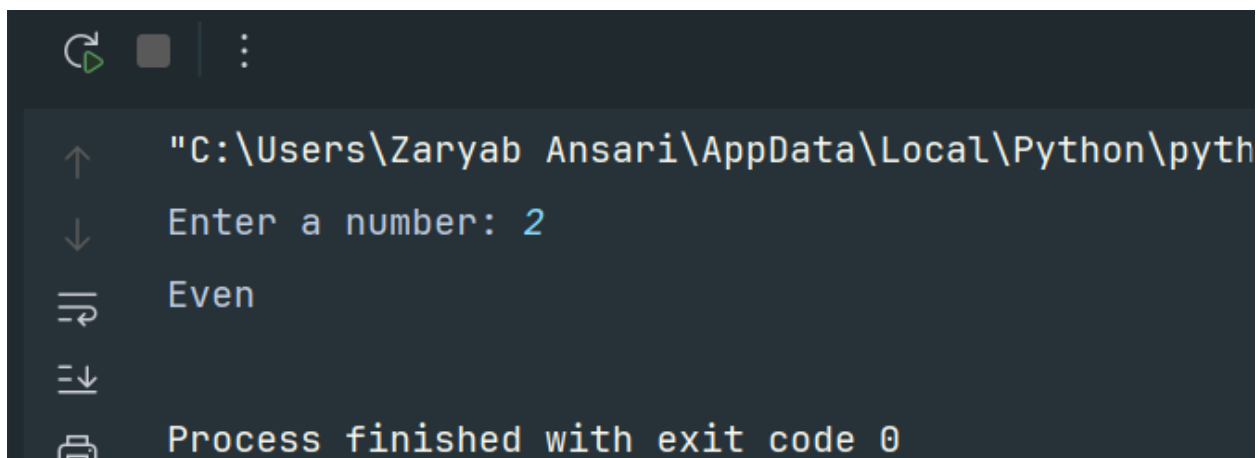
```
if num % 2 == 0:
```

```
    print("Even")
```

```
else:
```

```
    print("Odd")
```

OUTPUT:



```

C:\Users\Zaryab Ansari\AppData\Local\Python\python
Enter a number: 2
Even
Process finished with exit code 0

```

TASK 4:

```
a = int(input("Enter first number: "))
```

```
b = int(input("Enter second number: "))
```

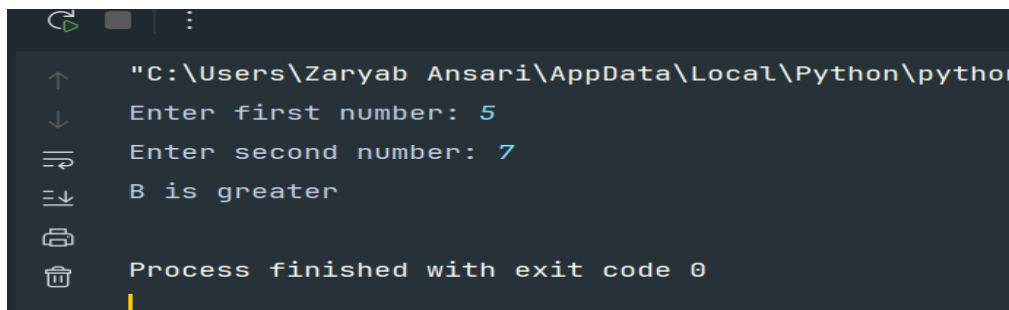
```
if a > b:
```

```
    print("A is greater")
```

```
else:
```

```
    print("B is greater")
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\AppData\Local\Python\python
Enter first number: 5
Enter second number: 7
B is greater
Process finished with exit code 0
```

TASK 5:

```
a = int(input("Enter number 1: "))
```

```
b = int(input("Enter number 2: "))
```

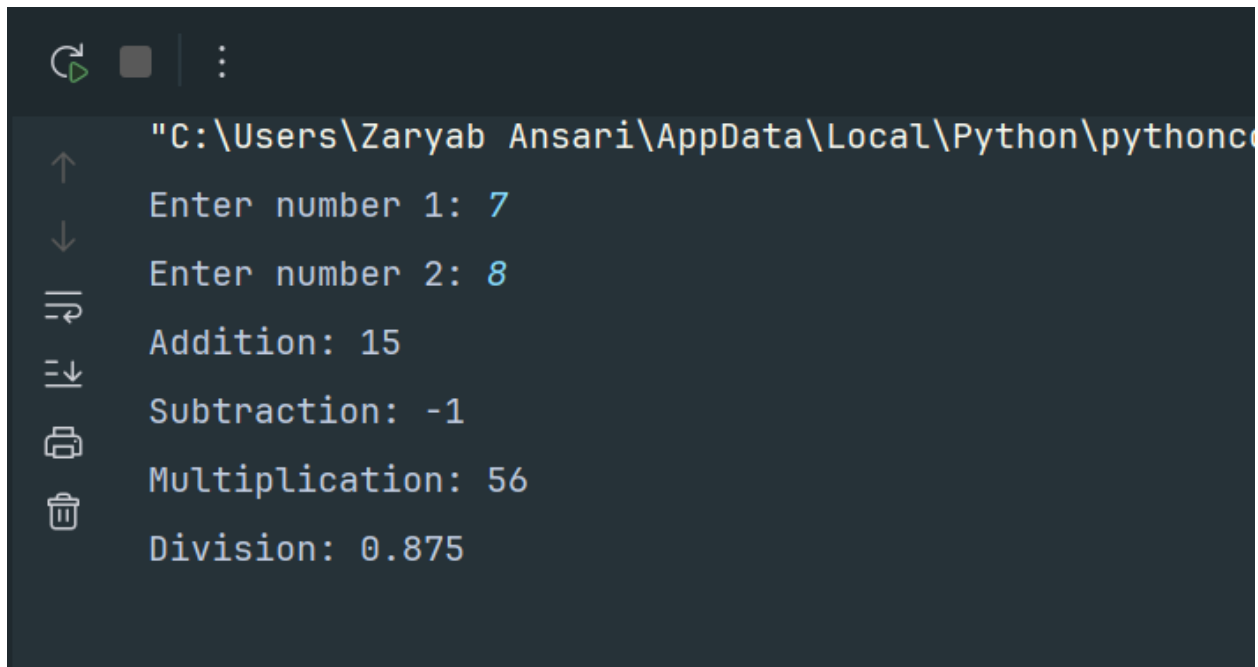
```
print("Addition:", a + b)
```

```
print("Subtraction:", a - b)
```

```
print("Multiplication:", a * b)
```

```
print("Division:", a / b)
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\AppData\Local\Python\pythonco
Enter number 1: 7
Enter number 2: 8
Addition: 15
Subtraction: -1
Multiplication: 56
Division: 0.875
```

Conclusion:

In this lab, we learned:

- How to take input and display output
- Basic data types
- Operators usage
- Decision making using if/else

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

3

LAB 2:

For Loop

OBJECTIVES:

- To understand the concept of iterative statements
- To learn how to use the for loop in Python
- To perform repetition using range and sequences
- To solve mathematical and logical problems using loopS

- To improve programming logic through iteration

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

A **for loop** in Python is used to execute a block of code repeatedly for a specific number of times. It is mainly used when the number of iterations is known.

The `range()` function is commonly used with for loops to generate a sequence of numbers.

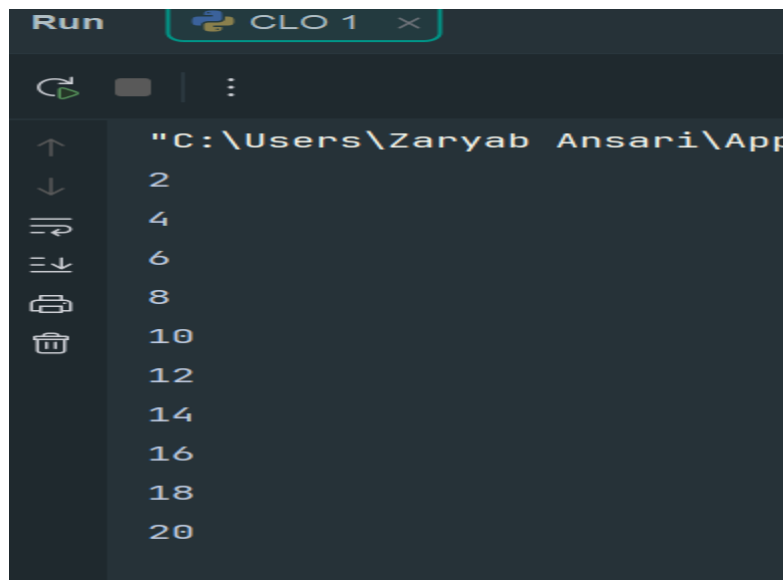
LAB WORK:

TASK 1:

```
for i in range(2, 21, 2):
```

```
    print(i)
```

OUTPUT:

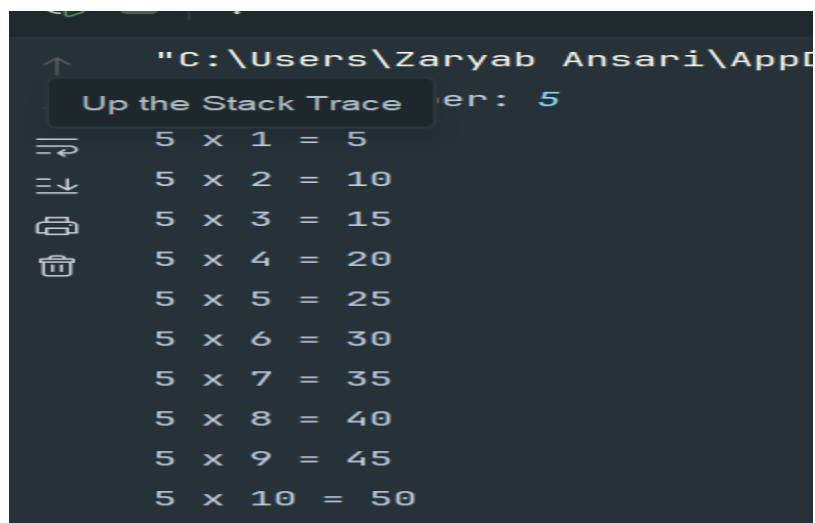


```
Run CLO 1 x
"C:\Users\Zaryab Ansari\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10.0.0_x-ww
2
4
6
8
10
12
14
16
18
20
```

TASK 2:

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

OUTPUT:



```
Run CLO 1 x
"C:\Users\Zaryab Ansari\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.10.0.0_x-ww
Up the Stack Trace Enter: 5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
```


TASK 3:

```
n = int(input("Enter a number: "))
```

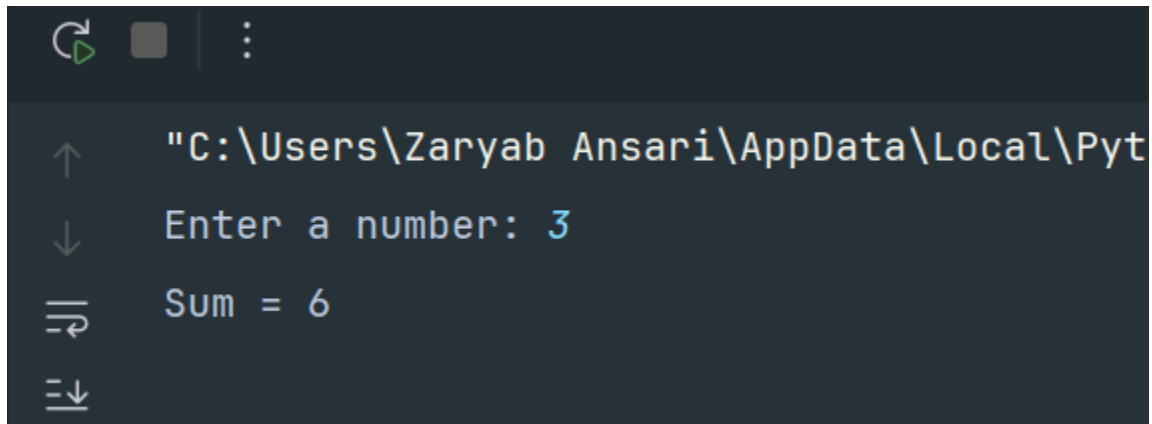
```
total = 0
```

```
for i in range(1, n + 1):
```

```
    total += i
```

```
print("Sum =", total)
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\AppData\Local\Python\Scripts\python.exe"
↓ Enter a number: 3
⇐ Sum = 6
⇓
```

TASK 4:

```
text = "Python"
```

```
for ch in text:
```

```
    print(ch)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\AppData\Local\Microsoft\Windows\CurrentVersion\FontEulerCache\
↓ P
⇌ y
⇌ t
🖨 h
🗑 o
n

Process finished with exit code 0
```

CONCLUSION:

In this lab, we studied the working of the for loop in Python. We used it to perform repetitive tasks like printing numbers, generating tables, and calculating sums. It helps make programs shorter, cleaner, and more efficient.

**University Of Engineering and Technology, Lahore
Computer Engineering Department**

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 3:

_While Loop

OBJECTIVES:

- To understand the concept of conditional looping
- To learn how to use the `while` loop in Python
- To execute a block of code based on a condition
- To solve problems where the number of iterations is unknown
- To develop logical thinking using loops

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)

- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

A **while loop** in Python is used to repeatedly execute a block of code as long as a given condition is true.

Unlike the **for** loop, the **while** loop is used when the number of iterations is **not fixed**.

Syntax:

while condition:

 # code block

LAB WORK:

TASK 1:

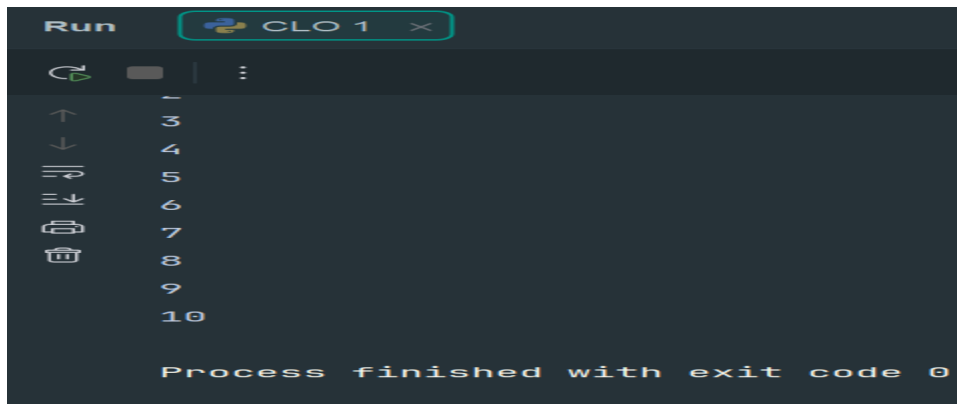
```
i = 1
```

```
while i <= 10:
```

```
    print(i)
```

```
    i += 1
```

OUTPUT:



```
1
2
3
4
5
6
7
8
9
10

Process finished with exit code 0
```

TASK 2:

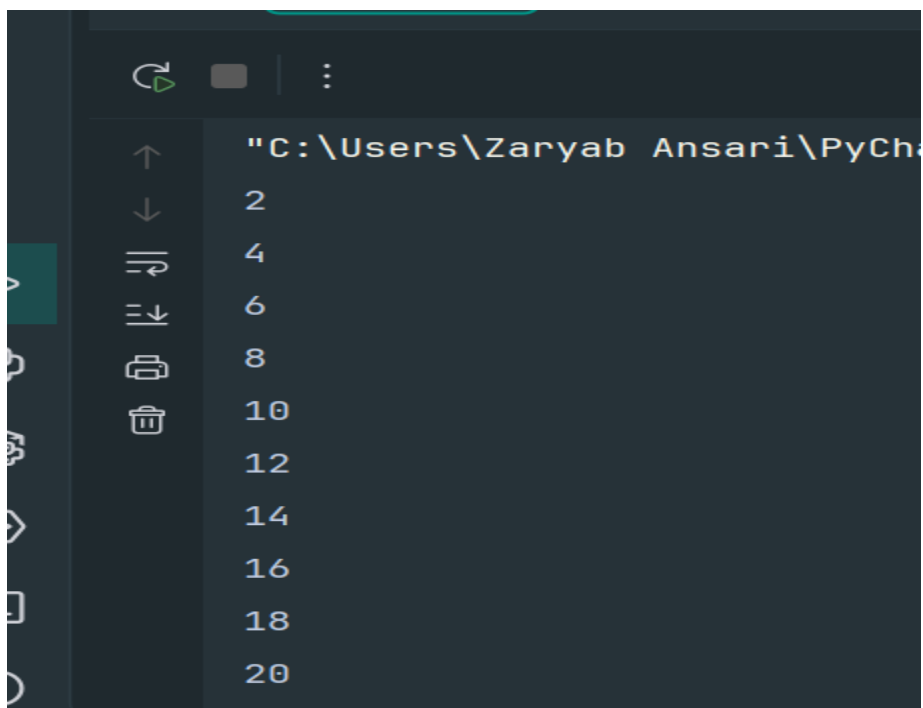
```
i = 2
```

```
while i <= 20:
```

```
    print(i)
```

```
    i += 2
```

OUTPUT:



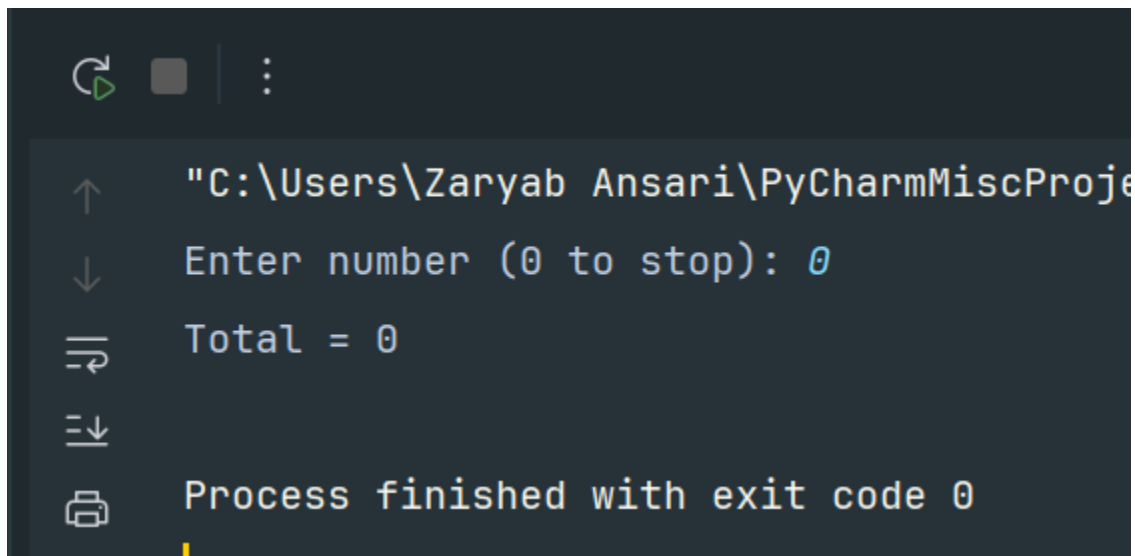
```
"C:\Users\Zaryab Ansari\PyCharm\bin\python.exe"
2
4
6
8
10
12
14
16
18
20

Process finished with exit code 0
```

TASK 3:

```
total = 0
num = int(input("Enter number (0 to stop): "))
while num != 0:
    total += num
    num = int(input("Enter number (0 to stop): "))
print("Total =", total)
```

OUTPUT:



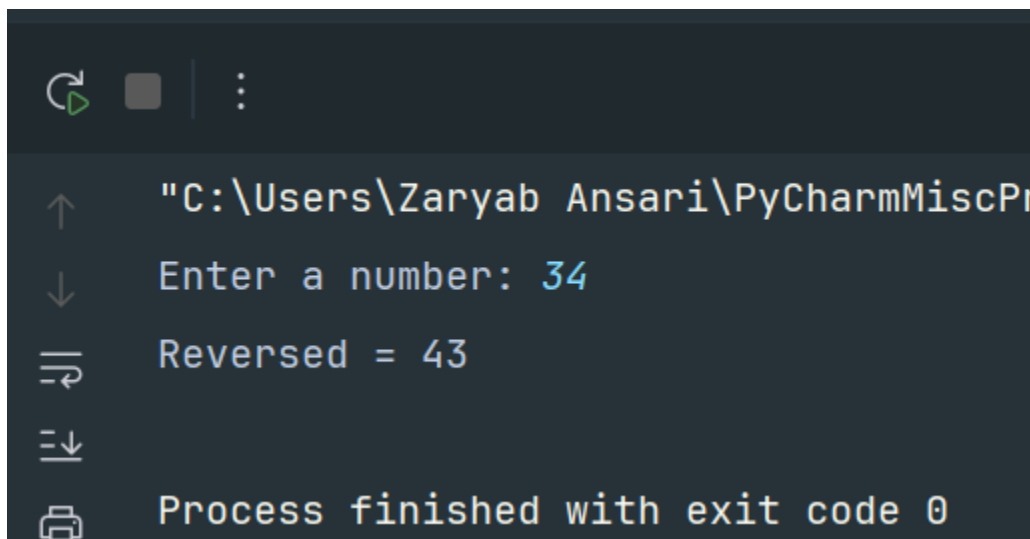
```
"C:\Users\Zaryab Ansari\PyCharmMiscProje
Enter number (0 to stop): 0
Total = 0
Process finished with exit code 0
```

TASK 4:

```
num = int(input("Enter a number: "))
rev = 0
```

```
while num > 0:  
    digit = num % 10  
    rev = rev * 10 + digit  
    num = num // 10  
print("Reversed =", rev)
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMiscPr  
Enter a number: 34  
Reversed = 43  
Process finished with exit code 0
```

CONCLUSION:

In this lab, we learned how the while loop works based on conditions. We used it in different problems like summation, reversing numbers, and tables. It is very useful when the number of repetitions is not known in advance.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 4:

NESTED LOOPS

OBJECTIVE:

The objective of this lab is to understand and implement **nested loops** in Python. Students will learn how to use loops inside other loops to solve problems involving patterns, tables, and repeated structures

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)

- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

A **nested loop** is a loop inside another loop. The inner loop executes completely for every iteration of the outer loop.

Nested loops are useful in:

- Pattern printing
- Matrix operations
- Table generation
- Multi-dimensional data handling

Syntax:

```
for outer_variable in range():
```

```
    for inner_variable in range():
```

```
        # code block
```

Working:

- The outer loop runs first
- For each iteration of the outer loop, the inner loop runs completely
- Total iterations = outer loop $\times$ inner loop

LAB WORK:

TASK 1:

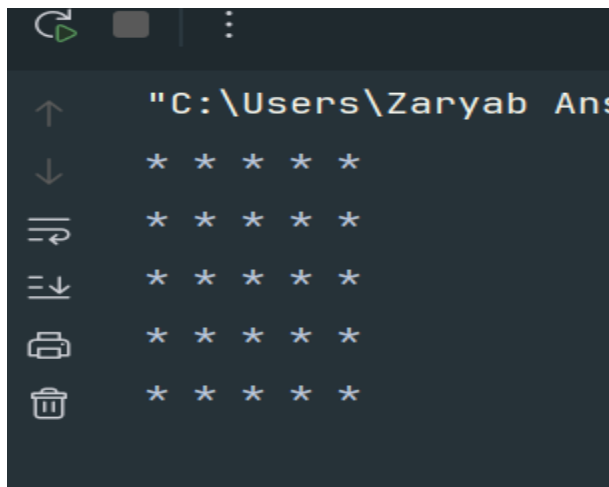
```
for i in range(5):
```

```
    for j in range(5):
```

```
        print("*", end=" ")
```

```
    print()
```

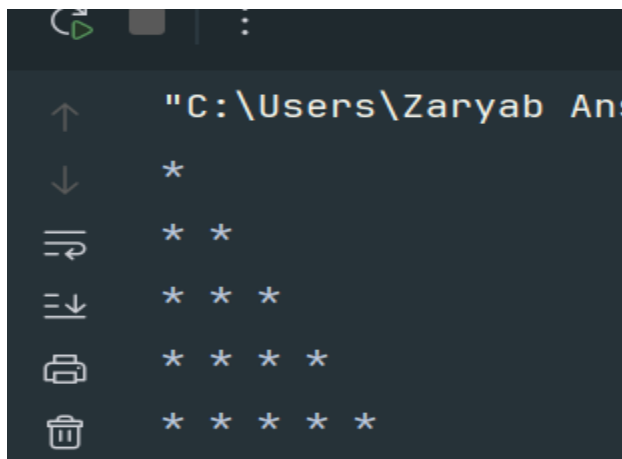
OUTPUT:



TASK 2:

```
for i in range(1, 6):  
    for j in range(i):  
        print("*", end=" ")  
    print()
```

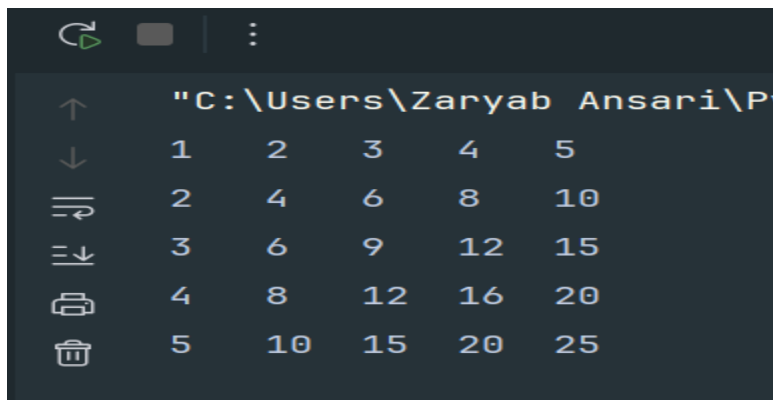
OUTPUT:



TASK 3:

```
for i in range(1, 6):  
    for j in range(1, 6):  
        print(i * j, end="\t")  
    print()
```

OUTPUT:

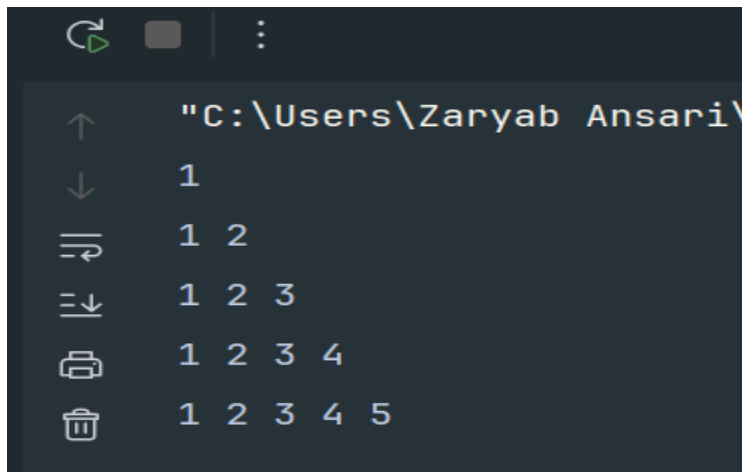


```
↑ "C:\Users\Zaryab Ansari\Py  
↓ 1 2 3 4 5  
↺ 2 4 6 8 10  
↻ 3 6 9 12 15  
🖨 4 8 12 16 20  
🗑 5 10 15 20 25
```

TASK 4:

```
for i in range(1, 6):  
    for j in range(1, i+1):  
        print(j, end=" ")  
    print()
```

OUTPUT:

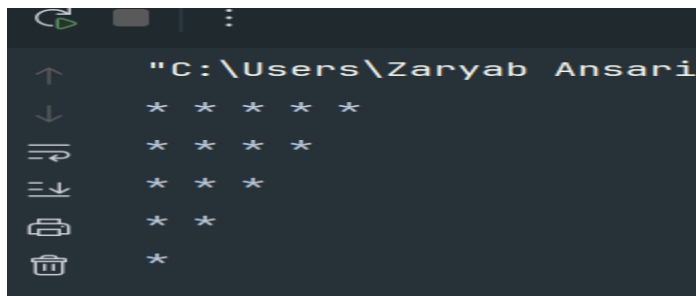


```
↑ "C:\Users\Zaryab Ansari\  
↓ 1  
⇌ 1 2  
⇌ 1 2 3  
⇌ 1 2 3 4  
⇌ 1 2 3 4 5
```

TASK 5:

```
for i in range(5, 0, -1):  
    for j in range(i):  
        print("*", end=" ")  
    print()
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\  
↓ * * * * *  
⇌ * * * *  
⇌ * * *  
⇌ * *  
⇌ *
```

CONCLUSION:

Nested loops are essential for solving problems that involve repetition within repetition. They are widely used in pattern generation, tables, and complex data structures, helping students build strong logical and programming skills.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

Lab 1:

Functions in Python

Objectives:

- To understand the concept of functions in Python
- To learn how to define and call functions
- To use parameters and arguments in functions
- To understand return values from functions
- To develop problem-solving skills using functions

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)

- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

Theory:

A **function** in Python is a block of organized and reusable code used to perform a single task. Functions help in reducing code repetition and make programs easier to understand and maintain.

Functions are defined using the **def** keyword followed by the function name and parentheses.

There are different types of functions:

- **Built-in functions** (e.g., `print()`, `len()`)
- **User-defined functions**

Functions may:

- Take **parameters (inputs)**
- Return **values using return statement**

Advantages:

- Code reusability
- Better organization
- Easier debugging

Lab WORK:

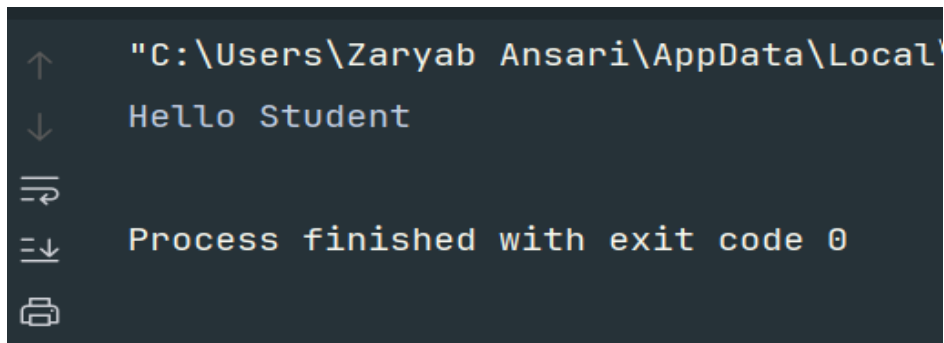
TASK 1:

```
def greet():
```

```
    print("Hello Student")
```

```
greet()
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\AppData\Local\
Hello Student
Process finished with exit code 0
```

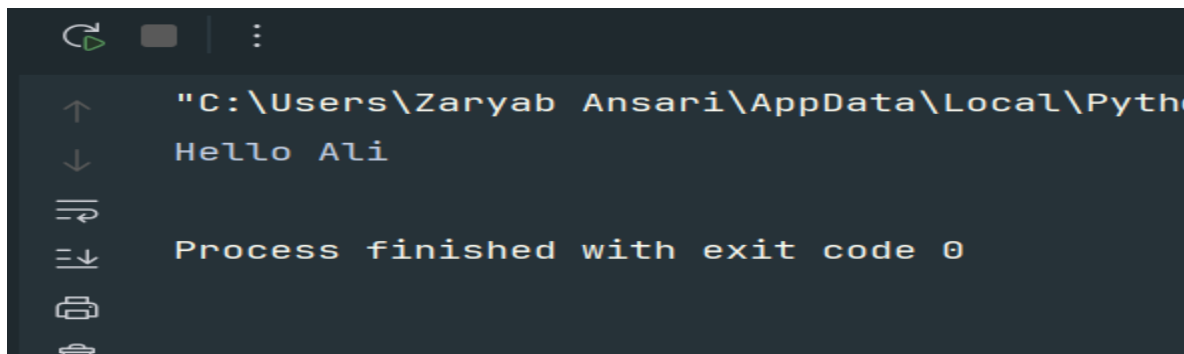
TASK 2:

def greet(name):

 print("Hello", name)

greet("Ali")

OUTPUT:



```
"C:\Users\Zaryab Ansari\AppData\Local\Python
Hello Ali
Process finished with exit code 0
```

TASK 3:

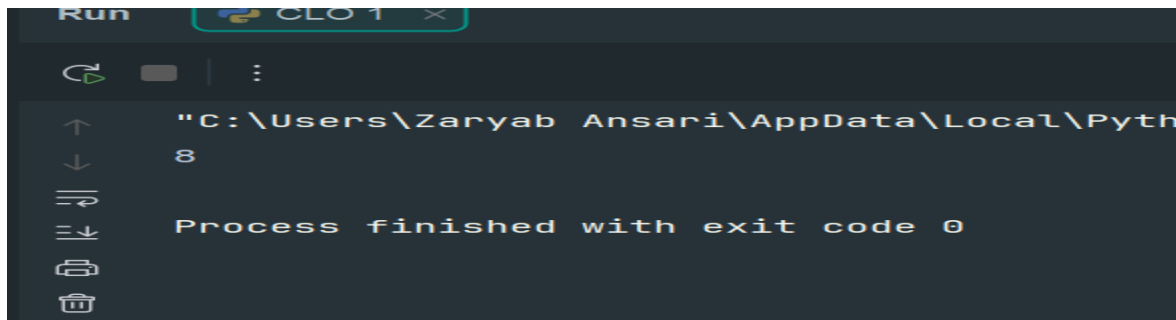
def add(a, b):

 return a + b

result = add(5, 3)

print(result)

OUTPUT:

A screenshot of a Python IDE's output window. The window has a title bar with 'Run' and 'CLO 1'. It contains a green play button icon, a stop button icon, and a menu icon. The output text is: `"C:\Users\Zaryab Ansari\AppData\Local\Python\Python38\python.exe"`, `8`, and `Process finished with exit code 0`.

```
Run CLO 1 x
"\"C:\Users\Zaryab Ansari\AppData\Local\Python\Python38\python.exe\"
8
Process finished with exit code 0
```

TASK 4:

def factorial(n):

 fact = 1

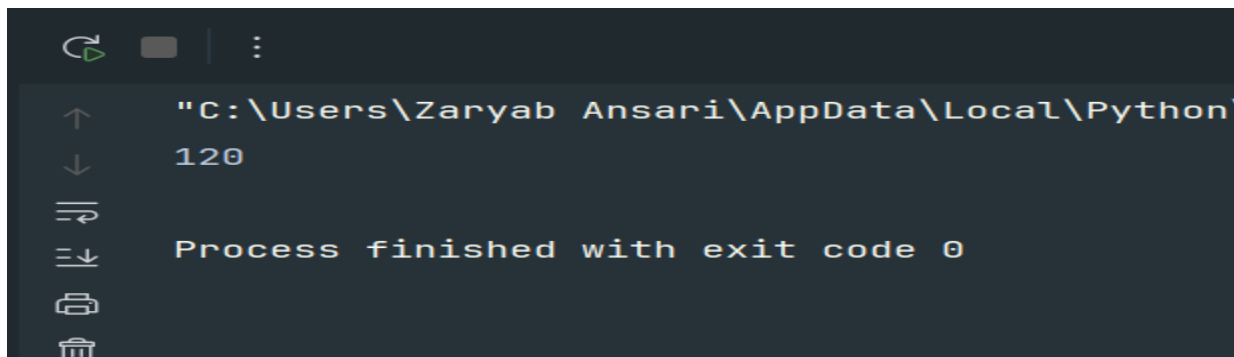
 for i in range(1, n + 1):

 fact *= i

 return fact

print(factorial(5))

OUTPUT:

A screenshot of a Python IDE's output window. The window has a title bar with 'Run' and 'CLO 1'. It contains a green play button icon, a stop button icon, and a menu icon. The output text is: `"C:\Users\Zaryab Ansari\AppData\Local\Python\Python38\python.exe"`, `120`, and `Process finished with exit code 0`.

```
Run CLO 1 x
"\"C:\Users\Zaryab Ansari\AppData\Local\Python\Python38\python.exe\"
120
Process finished with exit code 0
```

TASK 5:

def is_prime(n):

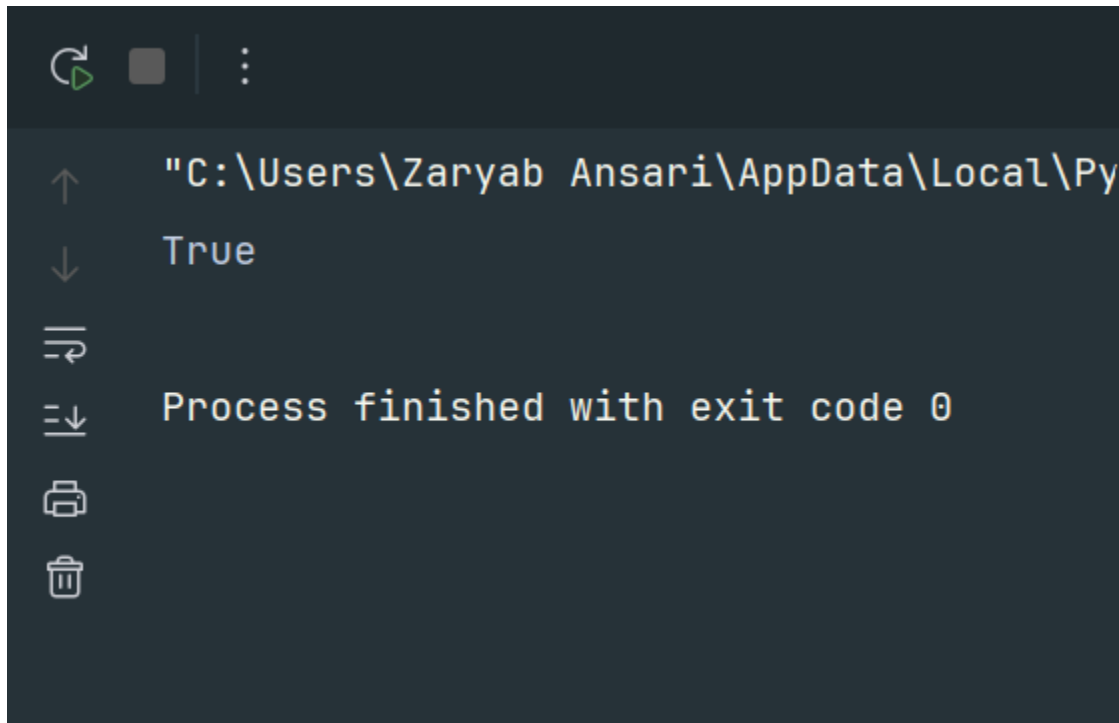
 if n < 2:

 return False

 for i in range(2, n):


```
        if n % i == 0:
            return False
    return True
print(is_prime(7))
```

OUTPUT:

A screenshot of a Python IDE window. The window has a dark theme. At the top, there's a toolbar with icons for running, stopping, and debugging. Below the toolbar, the output console shows the result of the program execution. The first line is the file path: "C:\Users\Zaryab Ansari\AppData\Local\Py...". The second line is the output of the program: True. The third line is the message: Process finished with exit code 0. On the left side of the output console, there are several icons: an upward arrow, a downward arrow, a refresh icon, a search icon, a print icon, and a trash icon.

Conclusion:

This lab helped in understanding the concept of variable scope in Python. We learned the difference between local and global variables and how to use the global keyword. Proper understanding of scope helps in writing error-free program

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 6:

LOCAL AND GLOBAL VARIABLES

OBJECTIVE:

the objective of this lab is to understand the concept of **local and global variables** in python. students will learn how variable scope works and how variables can be accessed inside and outside functions.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)

- basic knowledge of programming concepts

THEORY:

in python, variables are classified based on where they are declared and used.

1. local variables:

- declared inside a function
- can only be used within that function
- not accessible outside

2. global variables:

- declared outside all functions
- can be accessed anywhere in the program
- can be modified using `global` keyword

LAB WORK:

TASK 1:

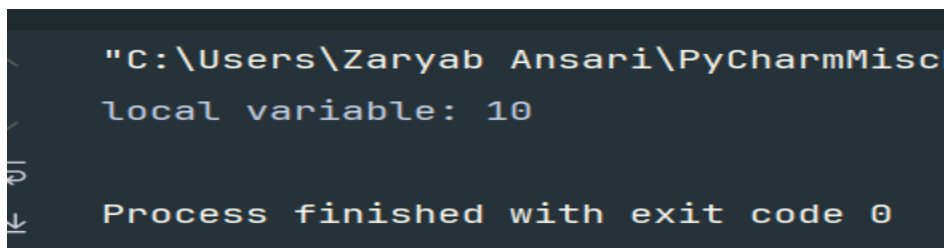
```
def show():
```

```
    x = 10
```

```
    print("local variable:", x)
```

```
show()
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMisc
local variable: 10

Process finished with exit code 0
```

TASK 2:

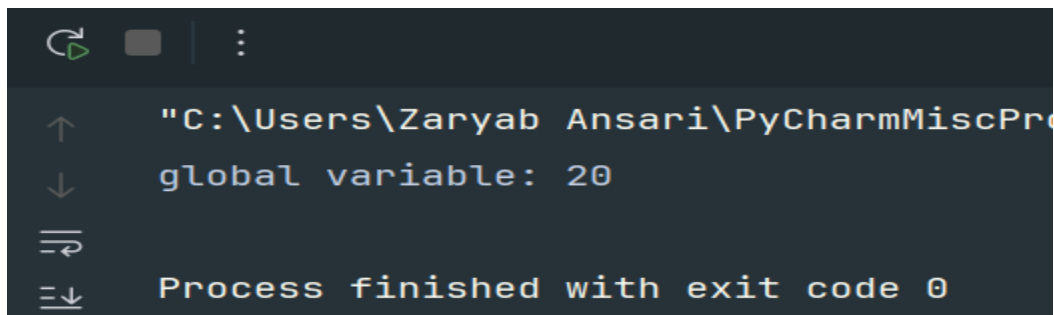
```
x = 20
```

```
def display():
```

```
    print("global variable:", x)
```

```
display()
```

OUTPUT:

A screenshot of a terminal window with a dark background. At the top, there are icons for a refresh button, a stop button, and a menu button. Below these, the terminal shows the file path "C:\Users\Zaryab Ansari\PyCharmMiscPro" followed by the output "global variable: 20". At the bottom, it says "Process finished with exit code 0". On the left side of the terminal, there are navigation icons: an up arrow, a down arrow, a double left arrow, and a double right arrow.

TASK 3:

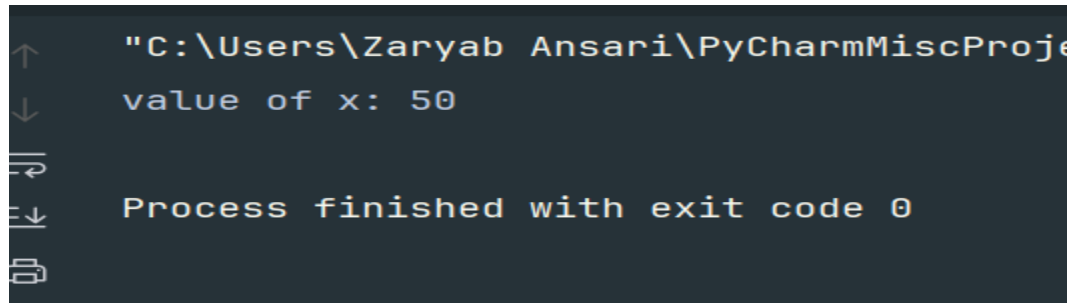
```
x = 50
```

```
def test():
```

```
    print("value of x:", x)
```

```
test()
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMiscProje
value of x: 50
Process finished with exit code 0
```

TASK 4:

```
x = 10
```

```
def change():
```

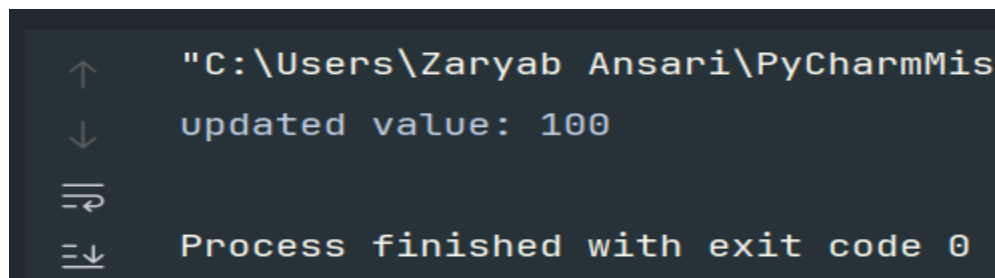
```
    global x
```

```
    x = 100
```

```
change()
```

```
print("updated value:", x)
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMisc
updated value: 100
Process finished with exit code 0
```

TASK 5:

```
x = 5
```

```
def check():
```

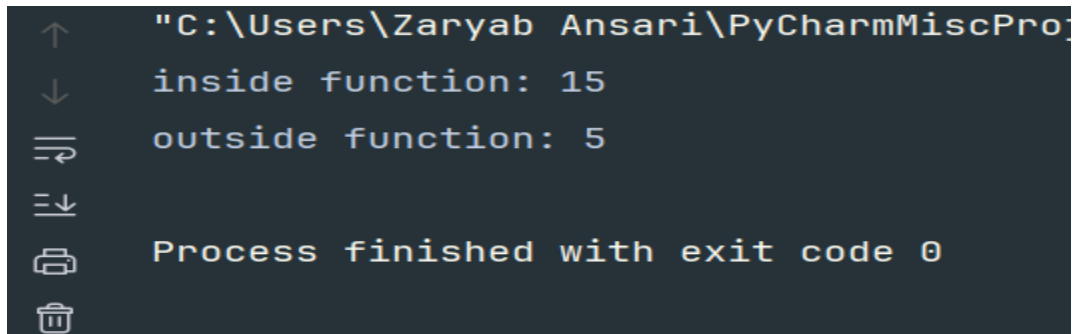
```
    x = 15
```

```
    print("inside function:", x)
```

```
check()
```

```
print("outside function:", x)
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMiscProj
inside function: 15
outside function: 5
Process finished with exit code 0
```

CONCLUSION:

this lab helps in understanding variable scope in python. local variables are limited to functions, while global variables can be accessed throughout the program. proper use of variable scope helps in writing clean and error-free code

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 7:

LIST AND TUPLE

OBJECTIVE:

the objective of this lab is to understand the concept of **lists and tuples** in python. students will learn how to store multiple values, access elements, and perform basic operations on these data structures.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)

- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

in python, lists and tuples are used to store multiple items in a single variable.

1. list:

- ordered and changeable (mutable)
- allows duplicate values
- defined using square brackets []

2. tuple:

- ordered but unchangeable (immutable)
- allows duplicate values
- defined using parentheses ()

difference:

- list → mutable
- tuple → immutable

LAB WORK:

TASK 1:

```
numbers = [1, 2, 3, 4, 5]
```

```
print("list:", numbers)
```

OUTPUT:


```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscPro
↓ list: [1, 2, 3, 4, 5]
||
=↓ Process finished with exit code 0
```

TASK 2:

```
fruits = ["apple", "banana", "cherry"]
print("first element:", fruits[0])
print("last element:", fruits[-1])
```

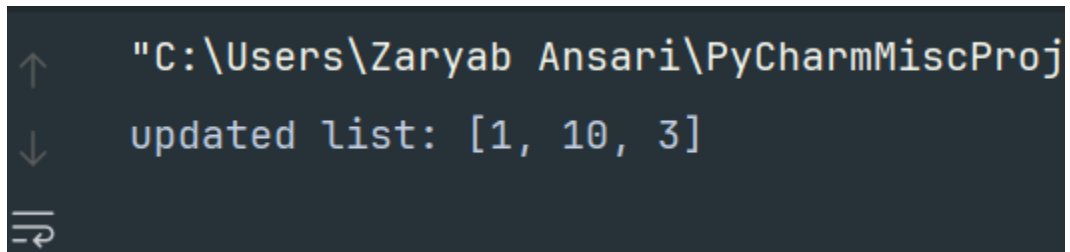
OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMisc
↓ first element: apple
||
=↓ last element: cherry
-↓
```

TASK 3:

```
numbers = [1, 2, 3]
numbers[1] = 10
print("updated list:", numbers)
```

OUTPUT:

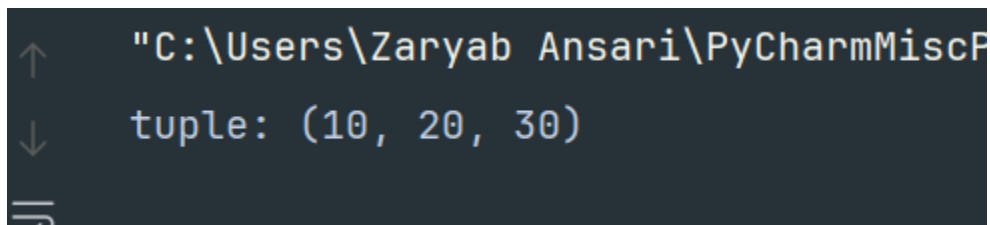
A terminal window with a dark background. The first line shows a file path: "C:\Users\Zaryab Ansari\PyCharmMiscProj". The second line shows the text "updated list: [1, 10, 3]". On the left side of the terminal, there are three icons: an upward arrow, a downward arrow, and a magnifying glass.

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProj
↓ updated list: [1, 10, 3]
```

TASK 4:

```
my_tuple = (10, 20, 30)
print("tuple:", my_tuple)
```

OUTPUT:

A terminal window with a dark background. The first line shows a file path: "C:\Users\Zaryab Ansari\PyCharmMiscP". The second line shows the output of a print statement: "tuple: (10, 20, 30)". On the left side of the terminal, there are three icons: an upward arrow, a downward arrow, and a magnifying glass.

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscP
↓ tuple: (10, 20, 30)
```

TASK 5:

```
my_tuple = (5, 10, 15)
print("first value:", my_tuple[0])
print("second value:", my_tuple[1])
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscP
↓ first value: 5
⇐ second value: 10
⇓
Process finished with exit code 0
```

CONCLUSION:

this lab helps in understanding how to store and manage multiple values using lists and tuples. lists are flexible and can be modified, while tuples provide fixed and secure data storage. both are important for efficient data handling in python

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 8:

SETS AND DICTIONARY

OBJECTIVE:

the objective of this lab is to understand the concept of **sets and dictionaries** in python. students will learn how to store unique data using sets and key-value pairs using dictionaries.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

in python, sets and dictionaries are important data structures used for storing and organizing data.

1. sets:

- unordered collection
- does not allow duplicate values
- defined using curly braces { }
- useful for unique elements

2. dictionary:

- stores data in key-value pairs
- keys must be unique
- defined using {key: value}

difference:

- set → stores only values (unique)
- dictionary → stores key-value pairs

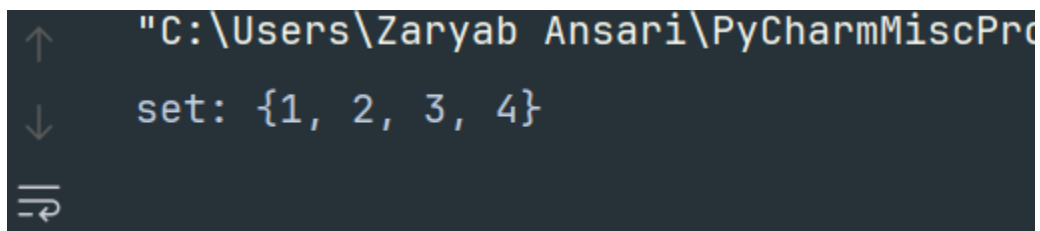
LAB WORK:

TASK 1:

```
numbers = {1, 2, 3, 4, 4}
```

```
print("set:", numbers)
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscPro  
↓ set: {1, 2, 3, 4}  
≡ ↻
```

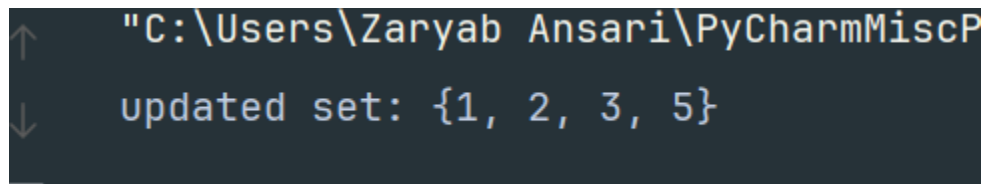
TASK 2:

```
numbers = {1, 2, 3}
```

```
numbers.add(5)
```

```
print("updated set:", numbers)
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscP  
↓ updated set: {1, 2, 3, 5}
```

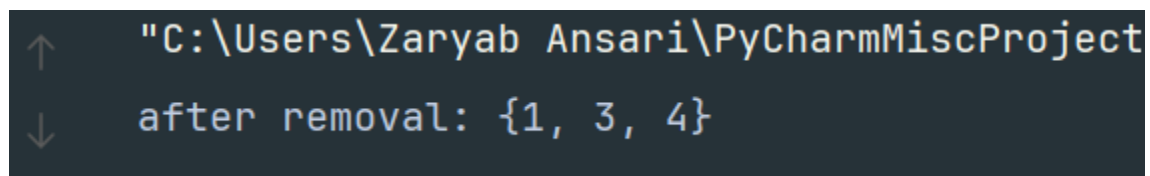
TASK 3:

```
numbers = {1, 2, 3, 4}
```

```
numbers.remove(2)
```

```
print("after removal:", numbers)
```

OUTPUT:

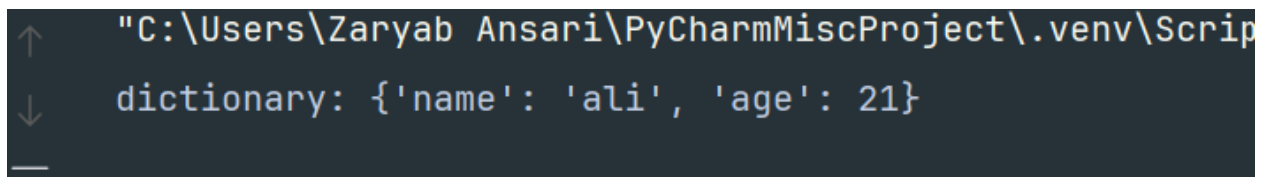


```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject  
↓ after removal: {1, 3, 4}
```

TASK 4:

```
student = {"name": "ali", "age": 21}
print("dictionary:", student)
```

OUTPUT:

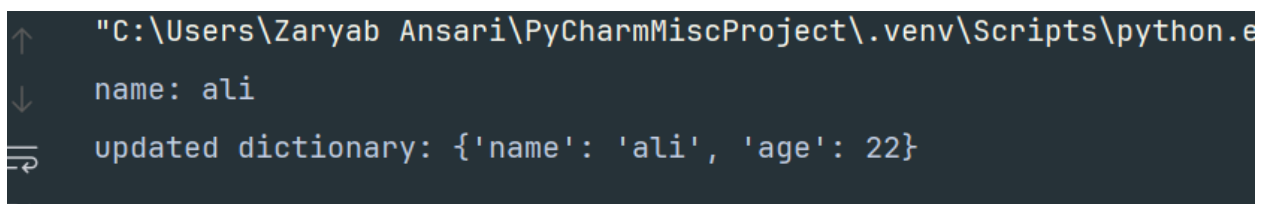
A terminal window with a dark background. The path "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe" is at the top. Below it, the output "dictionary: {'name': 'ali', 'age': 21}" is displayed. Navigation icons (up, down, search) are visible on the left side of the terminal.

```
"C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe
dictionary: {'name': 'ali', 'age': 21}
```

TASK 5:

```
student = {"name": "ali", "age": 21}
print("name:", student["name"])
student["age"] = 22
print("updated dictionary:", student)
```

OUTPUT:

A terminal window with a dark background. The path "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe" is at the top. Below it, the output "name: ali" is displayed on one line, and "updated dictionary: {'name': 'ali', 'age': 22}" is displayed on the next line. Navigation icons (up, down, search) are visible on the left side of the terminal.

```
"C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe
name: ali
updated dictionary: {'name': 'ali', 'age': 22}
```

CONCLUSION:

this lab helps in understanding how to store unique values using sets and structured data using dictionaries. sets are useful for removing duplicates, while dictionaries are useful for storing related data using keys and values.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 9:

ECAT PROJECT

OBJECTIVE:

the objective of this project is to design and develop a **problem-solving application using python**. the project aims to enhance programming skills by applying concepts such as variables, loops, functions, and data structures to solve real-world problems. it also focuses on improving logical thinking, code organization, and debugging skills.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection for research and troubleshooting
- required python libraries (if any, depending on project)

- basic understanding of programming concepts

THEORY:

INTRODUCTION TO PYTHON PROGRAMMING:

python is a high-level, interpreted programming language known for its simplicity and readability. it supports multiple programming paradigms, including procedural, object-oriented, and functional programming, making it suitable for a wide range of applications.

PROBLEM ANALYSIS AND DESIGN:

before implementing the project, the problem is carefully analyzed to understand the requirements. an algorithm or step-by-step solution is designed to ensure efficient and logical problem solving.

USE OF VARIABLES AND DATA TYPES:

variables are used to store data, while data types such as integers, floats, strings, and booleans define the type of data being handled. proper use of variables helps in managing and processing data effectively.

CONTROL STRUCTURES:

control structures such as conditional statements (**if-else**) and loops (**for, while**) are used to control the flow of the program. they allow decision-making and repetition of tasks based on specific conditions.

FUNCTIONS AND MODULAR PROGRAMMING:

functions are used to divide the program into smaller, reusable blocks. this improves code readability, reduces redundancy, and makes debugging easier.

DATA STRUCTURES:

data structures like lists, tuples, sets, and dictionaries are used to store and organize data efficiently. they help in handling large amounts of data in a structured manner.

ERROR HANDLING AND DEBUGGING:

exception handling is used to manage runtime errors and ensure smooth execution of the program. debugging techniques are applied to identify and fix errors during development.

APPLICATION DEVELOPMENT:

the project integrates all programming concepts to develop a complete application. it demonstrates how theoretical knowledge can be applied to solve real-world problems effectively.

TESTING AND IMPLEMENTATION:

after development, the program is tested with different inputs to ensure correctness and reliability. proper implementation ensures that the application meets all requirements.

MAIN CODE:

```

5 questions = [
6 {"q": "2+2=?", "A": "3", "B": "4", "C": "5", "D": "6", "ans": "B"},
7 {"q": "Capital of Pakistan?", "A": "Lahore", "B": "Karachi", "C": "Islamabad", "D": "Quetta", "ans": "C"},
8 {"q": "5*2=?", "A": "10", "B": "12", "C": "8", "D": "15", "ans": "A"},
9 {"q": "Sun rises from?", "A": "West", "B": "North", "C": "South", "D": "East", "ans": "D"},
10 {"q": "Python is?", "A": "Snake", "B": "Language", "C": "Game", "D": "Food", "ans": "B"},
11 {"q": "10/2=?", "A": "3", "B": "4", "C": "5", "D": "6", "ans": "C"},
12 {"q": "Water formula?", "A": "H2O", "B": "CO2", "C": "O2", "D": "NaCl", "ans": "A"},
13 {"q": "3^2=?", "A": "6", "B": "9", "C": "12", "D": "3", "ans": "B"},
14 {"q": "Earth is?", "A": "Star", "B": "Planet", "C": "Moon", "D": "Sun", "ans": "B"},
15 {"q": "Speed unit?", "A": "m/s", "B": "kg", "C": "m", "D": "s", "ans": "A"}
16 ]
17
18 all_results = []

```

```

23 def login(user, pwd): 2 usages
24     for i in range(3):
25         u = input("Username: ")
26         p = input("Password: ")
27         if u == user and p == pwd:
28             return True
29         else:
30             print("Wrong! Attempts left:", 2-i)
31     return False
32

```

```

def show_rules(): 1 usage
    print("\n📋 EXAM RULES")
    print("1. Each correct answer = +4 marks")
    print("2. Each wrong answer = -1 mark")
    print("3. You can skip using 'S'")
    print("4. Type 'SUBMIT' anytime to finish exam")
    print("5. No cheating 😊")

```

```

def start_exam(name, roll): 1 usage
    correct = 0
    wrong = 0
    skip = 0

```

```

for i, q in enumerate(questions):
    print("\nQ", i+1, q["q"])
    print("A)", q["A"], "B)", q["B"], "C)", q["C"], "D)", q["D"])

    ans = input("Answer (A/B/C/D/S/SUBMIT): ").upper()

    if ans == "SUBMIT":
        break

    if ans == "S":
        skip += 1
        continue

    if ans == q["ans"]:
        print("Correct ✅")
        correct += 1
    else:
        print("Wrong ❌")
        wrong += 1

```

```

69
70     score = correct * 4 - wrong
71     total = len(questions) * 4
72     percent = (score / total) * 100
73
74     if percent >= 80:
75         grade = "EXCELLENT"
76     elif percent >= 65:
77         grade = "GOOD"
78     elif percent >= 50:
79         grade = "AVERAGE"
80     else:
81         grade = "BELOW AVERAGE"
82
83     print("\n📌 RESULT")
84     print("Score:", score)
85     print("Percentage:", round(percent, 2))
86     print("Grade:", grade)

```

```

88         all_results.append({
89             "name": name,
90             "roll": roll,
91             "score": score,
92             "percent": percent,
93             "grade": grade
94         })
95
96
97
98
99     def view_questions(): 1 usage
100         for i, q in enumerate(questions):
101             print(i+1, q["q"], "| Ans:", q["ans"])
102
103
104     def add_question(): 1 usage
105         q = input("Enter question: ")
106         A = input("A: ")
107         B = input("B: ")
108         C = input("C: ")
109         D = input("D: ")
110         ans = input("Correct (A/B/C/D): ").upper()
111
112         questions.append({"q": q, "A": A, "B": B, "C": C, "D": D, "ans": ans})
113
114
115     def delete_question(): 1 usage
116         n = int(input("Enter question number: "))
117         if n <= len(questions):
118             questions.pop(n-1)
119
120
121     def view_results(): 1 usage
122         if not all_results:
123             print("No results yet!")
124             return
125
126         for r in all_results:
127             print("\n-----")
128             print("Name:", r["name"])
129             print("Roll:", r["roll"])
130             print("Score:", r["score"])
131             print("Percentage:", round(r["percent"], 2))
132             print("Grade:", r["grade"])
133

```

```

135 def search_student(): 1 usage
136     roll = input("Enter Roll Number: ")
137     found = False
138
139     for r in all_results:
140         if r["roll"] == roll:
141             print("\nStudent Found:")
142             print(r)
143             found = True
144
145     if not found:
146         print("No record found!")
147
148
149 def class_statistics(): 1 usage
150     if not all_results:
151         print("No results available!")
152         return
153
154     scores = [r["score"] for r in all_results]
155
156     print("\n📊 CLASS STATISTICS")
157     print("Highest:", max(scores))
158     print("Lowest:", min(scores))
159     print("Average:", round(sum(scores)/len(scores), 2))
160     pass_count = sum(1 for r in all_results if r["percent"] >= 50)
161     fail_count = len(all_results) - pass_count
162
163
164     print("Pass:", pass_count)
165     print("Fail:", fail_count)
166
167     grades = {"EXCELLENT":0, "GOOD":0, "AVERAGE":0, "BELOW AVERAGE":0}
168
169     for r in all_results:
170         grades[r["grade"]] += 1
171
172     print("\nGrade Distribution:")
173     for g in grades:
174         print(g, ":", grades[g])

```

```
if c == "1":
    view_questions()
elif c == "2":
    add_question()
elif c == "3":
    delete_question()
elif c == "4":
    view_results()
elif c == "5":
    search_student()
elif c == "6":
    class_statistics()
else:
    break
```

```

elif choice == "2":
    if login( user: "student", pwd: "student123"):
        name = input("Enter Name: ")
        roll = input("Enter Roll: ")

        show_rules()

        start = input("\nType 'START' to begin exam: ").upper()
        if start == "START":
            start_exam(name, roll)
        else:
            print("Exam not started.")

elif choice == "3":
    break

else:

```

OUTPUT:

```

↓
1. Admin Portal
2. Student Portal
3. Exit
Enter choice:

```

```

Enter choice: 2
Username: student
Password: student123
Enter Name: mishkat
Enter Roll: 6

```


📄 EXAM RULES

1. Each correct answer = +4 marks
2. Each wrong answer = -1 mark
3. You can skip using 'S'
4. Type 'SUBMIT' anytime to finish exam
5. No cheating 😊

Type 'START' to begin exam:

Q 1 2+2=?

A) 3 B) 4 C) 5 D) 6

Answer (A/B/C/D/S/SUBMIT): *B*

Correct ✅

CONCLUSION:

This project successfully demonstrates the application of fundamental programming concepts using python to solve a real-world problem. it helped in developing logical thinking, coding skills, and understanding of core concepts like loops, functions, and data structures. overall, the project meets its objectives and provides a strong foundation for future programming and software development.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 10:

RECURSION

OBJECTIVE:

the objective of this lab is to understand the concept of **recursion** in python. students will learn how a function can call itself to solve problems in a simpler and more efficient way.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)

- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

recursion is a programming technique where a function calls itself repeatedly until a stopping condition (base case) is met.

a recursive function has two main parts:

- **base case:** stops the recursion
- **recursive case:** function calls itself

recursion is useful in problems like:

- factorial calculation
- fibonacci series
- tree and graph traversal

syntax:

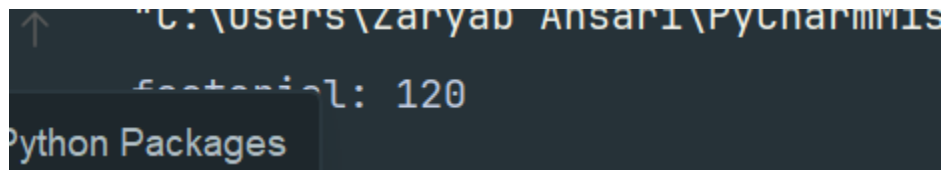
```
def function_name(parameters):  
    if base_condition:  
        return value  
  
    else:  
        return function_name(smaller_problem)
```

LAB WORK:

TASK 1:

```
def factorial(n):  
    if n == 1:  
        return 1  
    return n * factorial(n-1)  
  
print("factorial:", factorial(5))
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\PyCharm\Scripts
fibonacci: 120
Python Packages
```

TASK 2:

```
def fibonacci(n):
```

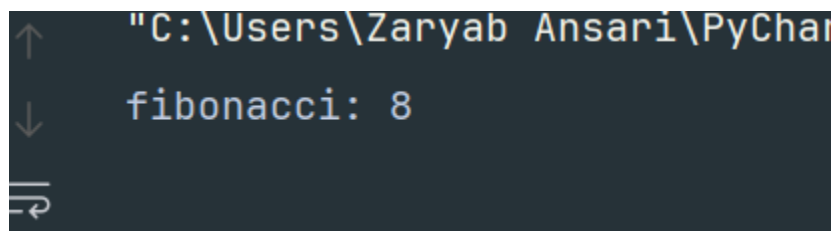
```
    if n <= 1:
```

```
        return n
```

```
    return fibonacci(n-1) + fibonacci(n-2)
```

```
print("fibonacci:", fibonacci(6))
```

OUTPUT:



```
↑ "C:\Users\Zaryab Ansari\PyCharm\Scripts
↓ fibonacci: 8
↺
```

TASK 3:

```
def sum_n(n):
```

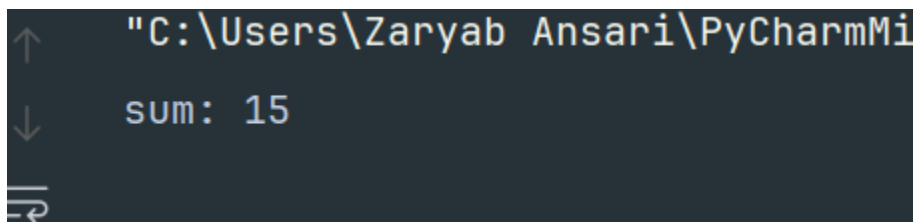
```
    if n == 1:
```

```
        return 1
```

```
    return n + sum_n(n-1)
```

```
print("sum:", sum_n(5))
```

OUTPUT:

A terminal window with a dark background. The first line shows a file path: "C:\Users\Zaryab Ansari\PyCharmMi. The second line shows the text "sum: 15". On the left side of the terminal, there are several navigation icons: an up arrow, a down arrow, and a refresh/circular arrow icon.

```
↑ "C:\Users\Zaryab Ansari\PyCharmMi
↓ sum: 15
↺
```

TASK 4:

```
def print_numbers(n):
```

```
    if n == 0:
```

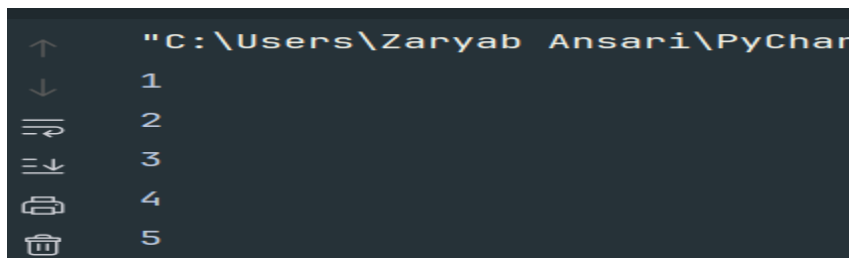
```
        return
```

```
    print_numbers(n-1)
```

```
    print(n)
```

```
print_numbers(5)
```

OUTPUT:

A terminal window with a dark background. The first line shows a file path: "C:\Users\Zaryab Ansari\PyChar. The subsequent lines show the numbers 1, 2, 3, 4, and 5, each on a new line. On the left side of the terminal, there are several navigation icons: an up arrow, a down arrow, a refresh/circular arrow icon, a search icon, a print icon, and a trash icon.

```
↑ "C:\Users\Zaryab Ansari\PyChar
↓ 1
↺ 2
↻ 3
🖨 4
🗑 5
```

TASK 5:

```
def power(base, exp):
```

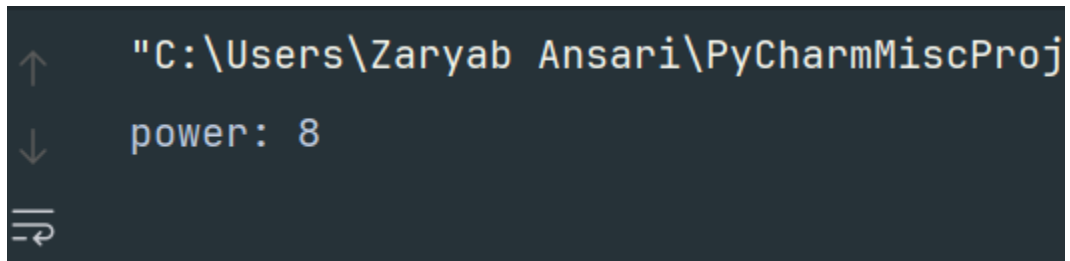
```
    if exp == 0:
```

```
        return 1
```

```
    return base * power(base, exp-1)
```

```
print("power:", power(2, 3))
```

OUTPUT:

A screenshot of a code editor with a dark background. On the left side, there are three icons: an upward arrow, a downward arrow, and a hamburger menu icon. The main area of the editor displays the text "C:\Users\Zaryab Ansari\PyCharmMiscProj" on the first line and "power: 8" on the second line, both in a light-colored monospace font.

CONCLUSION:

this lab helps in understanding recursion and how problems can be broken into smaller sub-problems. recursion simplifies complex problems but must always include a base case to avoid infinite loops.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 11:

EXCEPTION HANDLING

OBJECTIVE:

the objective of this lab is to understand the concept of **exception handling** in python. students will learn how to handle runtime errors and prevent program crashes using try and except blocks.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

exception handling is used to handle errors that occur during program execution.

common errors include:

- division by zero
- invalid input
- file not found

python provides the following keywords:

- `try` → block where error may occur
- `except` → handles the error
- `else` → runs if no error occurs
- `finally` → always executes

syntax:

`try:`

`# code that may cause error`

`except:`

`# code to handle error`

LAB WORK

TASK 1:

`try:`

`num = int(input("enter number: "))`

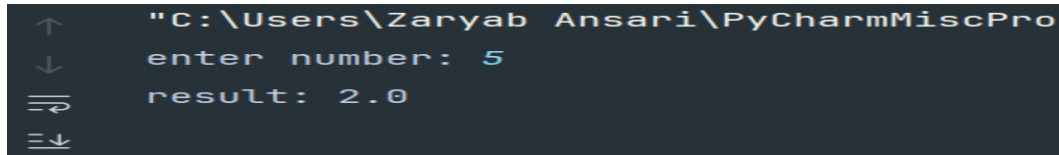
`result = 10 / num`

`print("result:", result)`

except:

```
print("cannot divide by zero")
```

OUTPUT:

A terminal window with a dark background. The prompt is "C:\Users\Zaryab Ansari\PyCharmMiscPro". The user enters "5" and the program outputs "result: 2.0".

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscPro
↓ enter number: 5
⇨ result: 2.0
⇩
```

TASK 2:

try:

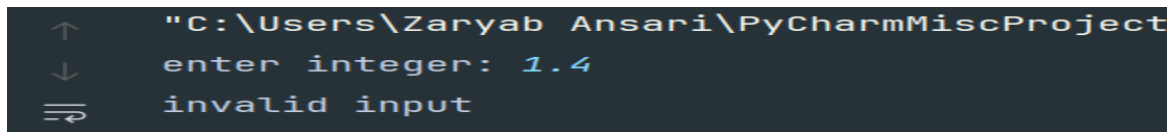
```
num = int(input("enter integer: "))
```

```
print("you entered:", num)
```

except:

```
print("invalid input")
```

OUTPUT:

A terminal window with a dark background. The prompt is "C:\Users\Zaryab Ansari\PyCharmMiscProject". The user enters "1.4" and the program outputs "invalid input".

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject
↓ enter integer: 1.4
⇨ invalid input
```

TASK 3:

try:

```
num = int(input("enter number: "))
```

except:

```
print("error")
```

else:

```
print("no error, value is:", num)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscPr
↓ enter number: 6
⇨ no error, value is: 6
⇩
```

TASK 4:

try:

```
num = int(input("enter number: "))
```

```
print(10 / num)
```

except:

```
print("error occurred")
```

finally:

```
print("this will always execute")
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\
↓ enter number: 7
⇨ 1.4285714285714286
⇩ this will always execute
```

TASK 5:

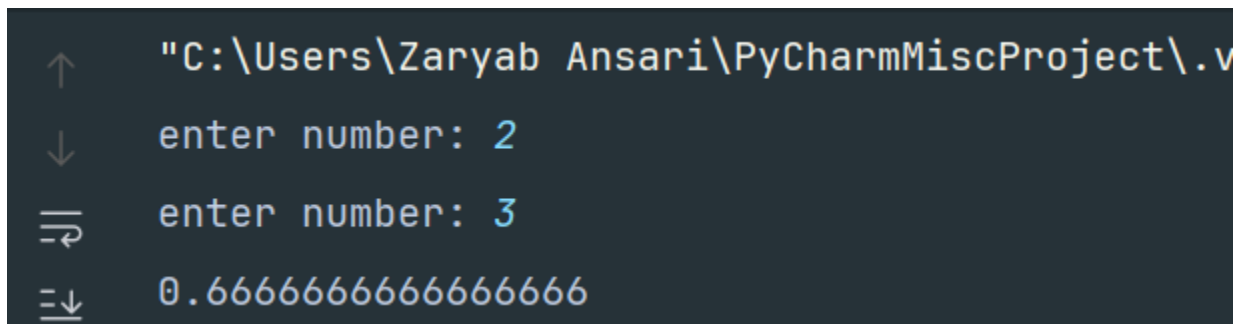
try:

```
a = int(input("enter number: "))
```

```
b = int(input("enter number: "))
```

```
    print(a / b)
except ZeroDivisionError:
    print("cannot divide by zero")
except ValueError:
    print("invalid input")
```

OUTPUT:

A screenshot of a terminal window with a dark background. It shows the execution of a Python script. The first line is a file path: "C:\Users\Zaryab Ansari\PyCharmMiscProject\...". The second line shows the prompt "enter number:" followed by the user input "2". The third line shows the prompt "enter number:" followed by the user input "3". The fourth line shows the output "0.6666666666666666".

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\...
↓ enter number: 2
⇨ enter number: 3
⇨↓ 0.6666666666666666
```

CONCLUSION:

this lab helps in understanding how to handle runtime errors using exception handling. it improves program reliability and prevents crashes by handling unexpected situations properly.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 12:

LAMBDA FUNCTIONS

OBJECTIVE:

the objective of this lab is to understand the concept of **lambda functions** in python. students will learn how to write short and simple functions using lambda expressions without defining a full function.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)

- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

a **lambda function** is a small anonymous function defined using the `lambda` keyword. it can have any number of arguments but only one expression.

lambda functions are useful when:

- a function is needed for a short time
- writing simple operations in one line
- working with functions like `map`, `filter`, etc.

syntax:

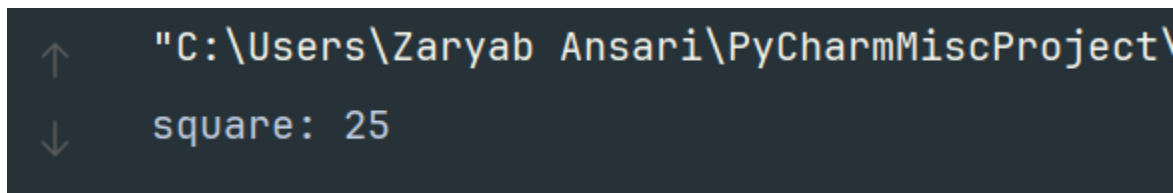
lambda arguments: expression

LAB WORK

TASK 1:

```
square = lambda x: x * x  
print("square:", square(5))
```

OUTPUT:



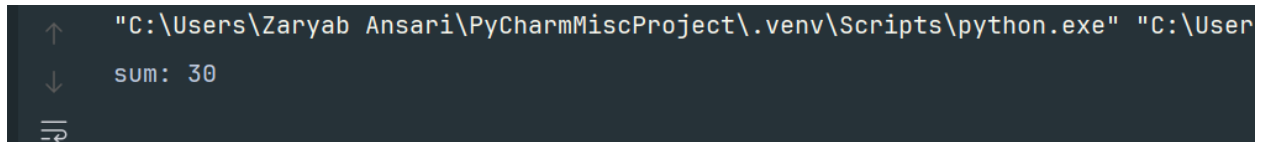
```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\  
↓ square: 25
```

TASK 2:

```
add = lambda a, b: a + b
```

```
print("sum:", add(10, 20))
```

OUTPUT:

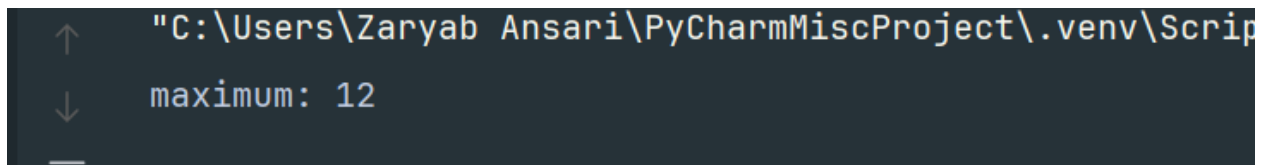


```
"C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe" "C:\User  
sum: 30
```

TASK 3:

```
maximum = lambda a, b: a if a > b else b  
print("maximum:", maximum(8, 12))
```

OUTPUT:



```
"C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scrip  
maximum: 12
```

TASK 4:

```
print("result:", (lambda x: x + 10)(5))
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python
↓ result: 15
||
```

TASK 5:

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))
print("updated list:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python
↓ updated list: [2, 4, 6, 8]
||
```

CONCLUSION:

this lab helps in understanding lambda functions and how they simplify small tasks. lambda functions make code shorter and more readable when used properly.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 13:

MAP, FILTER AND REDUCE

OBJECTIVE:

the objective of this lab is to understand the use of **map, filter, and reduce functions** in python. students will learn how to apply operations on collections of data efficiently using functional programming techniques.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)
- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)

- basic knowledge of programming concepts

THEORY:

python provides built-in functions to process data in lists and other collections.

1. map():

- applies a function to all items in an iterable
- returns a map object (can be converted to list)

2. filter():

- filters elements based on a condition
- returns only those elements that satisfy the condition

3. reduce():

- applies a function cumulatively to elements
- available in `functools` module

syntax:

`map(function, iterable)`

`filter(function, iterable)`

`reduce(function, iterable)`

LAB WORK

TASK 1:

```
numbers = [1, 2, 3, 4]
```

```
result = list(map(lambda x: x * 2, numbers))
```

```
print("mapped values:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\p
↓ mapped values: [2, 4, 6, 8]
=
```

TASK 2:

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
result = list(filter(lambda x: x % 2 == 0, numbers))
```

```
print("even numbers:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\p
↓ even numbers: [2, 4, 6]
=
```

TASK 3:

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(lambda a, b: a + b, numbers)
```

```
print("sum:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.v
↓ sum: 10
```

TASK 4:

```
numbers = [1, 2, 3, 4]
```

```
result = list(map(lambda x: x * x, numbers))
```

```
print("squares:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe
↓ squares: [1, 4, 9, 16]
```

TASK 5:

```
numbers = [-2, -1, 0, 1, 2, 3]
```

```
result = list(filter(lambda x: x > 0, numbers))
```

```
print("positive numbers:", result)
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe" "C:\Us
↓ positive numbers: [1, 2, 3]
```

CONCLUSION:

this lab helps in understanding how to process data efficiently using map, filter, and reduce. these functions make code shorter, cleaner, and more powerful when working with collections.

**University Of Engineering and Technology, Lahore
Computer Engineering Department**

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 14:

OS LIBRARY, NAME == "MAIN", IS VS ==

OBJECTIVE:

the objective of this lab is to understand the use of the **os library**, the importance of **if name == "main"**, and the difference between **is and == operators** in python.

EQUIPMENT REQUIRED:

- computer system (laptop or desktop)

- python installed (version 3.x)
- code editor or ide (such as visual studio code, pycharm, or idle)
- internet connection (optional for installation and help)
- basic knowledge of programming concepts

THEORY:

this lab covers three important concepts in python.

1. os library:

- used to interact with the operating system
- can create folders, get current directory, etc.

2. if name == "main":

- used to check whether the file is executed directly
- prevents code from running when imported as module

3. is vs ==:

- == checks value equality
- is checks memory location (identity)

LAB WORK

TASK 1:

```
import os
```

```
print("current directory:", os.getcwd())
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe
↓ current directory: C:\Users\Zaryab Ansari\PyCharmMiscProject
||
```

TASK 2:

```
import os
```

```
os.mkdir("test_folder")
```

```
print("folder created")
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\
↓ folder created
||
↳
⇒↓ Process finished with exit code 0
```

TASK 3:

```
def main(): print("this runs only when file is executed directly")
```

```
if name == "main": main()
```

OUTPUT:

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Scripts\python.exe
↓ this runs only when file is executed directly
||
↳
⇒↓ Process finished with exit code 0
```

TASK 4:

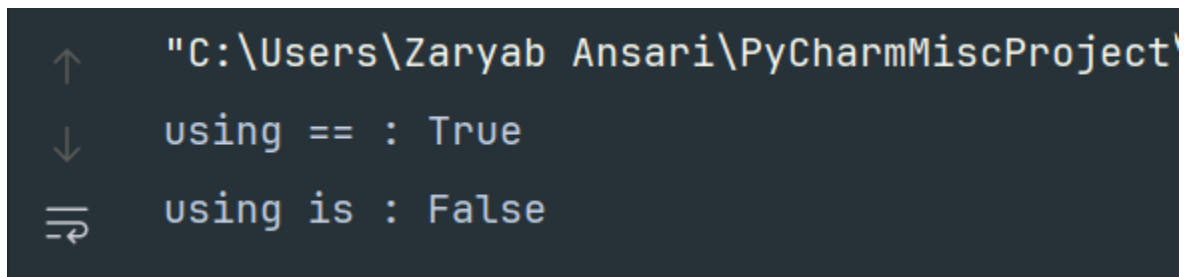
```
a = [1, 2, 3]
```

```
b = [1, 2, 3]
```

```
print("using == :", a == b)
```

```
print("using is :", a is b)
```

OUTPUT:

A terminal window with a dark background. The first line shows the file path "C:\Users\Zaryab Ansari\PyCharmMiscProject\" with a green cursor at the end. The second line shows the output "using == : True" with a green cursor at the end. The third line shows the output "using is : False" with a green cursor at the end.

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\  
↓ using == : True  
⇒ using is : False
```

TASK 5:

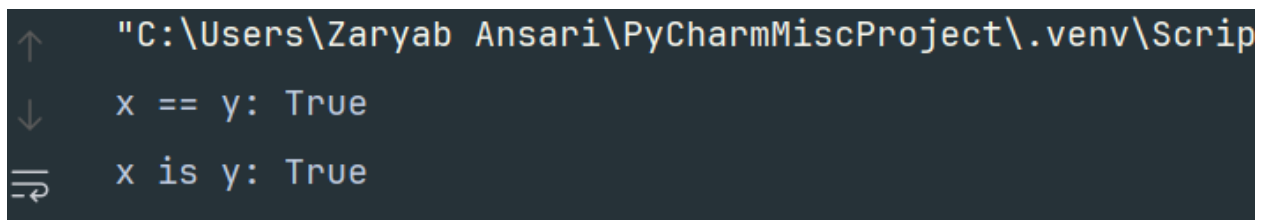
```
x = 10
```

```
y = 10
```

```
print("x == y:", x == y)
```

```
print("x is y:", x is y)
```

OUTPUT:

A terminal window with a dark background. The first line shows the file path "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Script\" with a green cursor at the end. The second line shows the output "x == y: True" with a green cursor at the end. The third line shows the output "x is y: True" with a green cursor at the end.

```
↑ "C:\Users\Zaryab Ansari\PyCharmMiscProject\.venv\Script\  
↓ x == y: True  
⇒ x is y: True
```


CONCLUSION:

this lab helps in understanding how to interact with the system using the os library, control program execution using the main function, and differentiate between equality and identity operators in python.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name:	Course Code: CMPE-112L
Assignment Type:	Dated:
Semester:	Session:
Lab/Project/Assignment #: 1	CLOs to be covered: CLO
Lab Title:	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB 15

INVENTORY MANAGEMENT SYSTEM

OBJECTIVES:

- To design and develop an Inventory Management System using **Python and PyQt (Qt Designer)**.
- To understand the use of **JSON for data storage and retrieval**.
- To implement CRUD operations (Create, Read, Update, Delete) for managing inventory items.
- To enhance GUI development skills using **Qt Designer**.
- To improve problem-solving and logical thinking through real-world application development.

EQUIPMENT REQUIRED:

- Computer/Laptop
- **Python (3.x)** installed
- **PyQt5 / PyQt6** library
- **Qt Designer** (for GUI design)
- Code Editor (e.g., VS Code / PyCharm)
- Basic knowledge of Python programming

THEORY:

Inventory Management System

An Inventory Management System is used to track, manage, and organize products, quantities, and related information efficiently. It helps in maintaining accurate records and reduces manual errors.

Python Programming

Python is a high-level, object-oriented programming language known for its simplicity and readability. It is widely used for developing applications, including GUI-based systems.

PyQt and Qt Designer

PyQt is a set of Python bindings for the Qt framework, used to create graphical user interfaces. Qt Designer allows users to visually design UI components such as buttons, labels, and tables, which can then be integrated into Python code.

JSON (JavaScript Object Notation)

JSON is a lightweight data format used for storing and exchanging data. In this project, JSON is used to store inventory records in a structured format, making it easy to read and update.

CRUD Operations

The system performs basic operations:

- **Create:** Add new items to inventory
- **Read:** Display stored inventory data
- **Update:** Modify existing item details
- **Delete:** Remove items from inventory

Event Handling in GUI

Event handling refers to responding to user actions like button clicks. PyQt allows connecting UI elements to functions that execute specific tasks when triggered.

MAIN CODE:

```
3  > import ...
8
9
10 def main(): 1 usage
11     app = QApplication(sys.argv)
12     window = InventoryWindow()
13     window.show()
14     app.exec_()
15     # sys.exit(app.exec_())
16
17
18 if __name__ == "__main__":
19     main()
20
```

GUI CODE:

```

> import ...

UI_FILE = Path(__file__).parent / "ui" / "main_window.ui"

class InventoryWindow(QMainWindow): 2 usages
    def __init__(self):
        super().__init__()

        uic.loadUi(str(UI_FILE), self)

        self.items = inventory_data.load_items()
        self.selected_id = None

```

```

4         self.setup_table()
5         self.connect_signals()
6         self.refresh_table()
7         self.statusbar.showMessage("Ready. Add your first item!")
8
9     def setup_table(self): 1 usage
10
11         self.inventoryTable.setColumnWidth(0, 50)
12         self.inventoryTable.setColumnWidth(1, 200)
13         self.inventoryTable.setColumnWidth(2, 100)
14         self.inventoryTable.setColumnWidth(3, 100)
15

```

```

6     def connect_signals(self): 1 usage
7
8         self.addButton.clicked.connect(self.add_item)
9         self.updateButton.clicked.connect(self.update_item)
10        self.deleteButton.clicked.connect(self.delete_item)
11        self.clearButton.clicked.connect(self.clear_form)
12        self.searchInput.textChanged.connect(self.refresh_table)
13        self.inventoryTable.itemSelectionChanged.connect(self.on_row_selected)
14

```

```

def refresh_table(self): 5 usages

    search_text = self.searchInput.text().strip().lower()

    if search_text:
        visible_items = [
            item for item in self.items
            if search_text in item["name"].lower()
        ]
    else:
        visible_items = self.items

    self.inventoryTable.setRowCount(len(visible_items))

```

```

for row, item in enumerate(visible_items):
    self.inventoryTable.setItem(row, 0, QTableWidgetItem(str(item["id"])))
    self.inventoryTable.setItem(row, 1, QTableWidgetItem(item["name"]))
    self.inventoryTable.setItem(row, 2, QTableWidgetItem(str(item["quantity"])))
    self.inventoryTable.setItem(row, 3, QTableWidgetItem(f"{item['price']:.2f}"))

```

```

def get_form_data(self): 2 usages

    name = self.nameInput.text().strip()
    quantity = self.quantityInput.value()
    price = self.priceInput.value()

    return name, quantity, price

```

```

def add_item(self): 1 usage

    name, quantity, price = self.get_form_data()

    if not name:
        QMessageBox.warning(self, title: "Missing Name", text: "Please enter a product name")
        return

    new_item = {
        "id": inventory_data.get_next_id(self.items),
        "name": name,
        "quantity": quantity,
        "price": price,
    }
    self.items.append(new_item)
    inventory_data.save_items(self.items)

```

```

self.clear_form()
self.refresh_table()
self.statusbar.showMessage(f"Added: {name}")

def update_item(self): 1 usage

    if self.selected_id is None:
        QMessageBox.information(self, title: "No Selection", text: "Select an item from the table first.")
        return

    name, quantity, price = self.get_form_data()

    if not name:
        QMessageBox.warning(self, title: "Missing Name", text: "Please enter a product name.")
        return

```

```

5         for item in self.items:
6             if item["id"] == self.selected_id:
7                 item["name"] = name
8                 item["quantity"] = quantity
9                 item["price"] = price
10                break
11
12        inventory_data.save_items(self.items)
13        self.refresh_table()
14        self.statusbar.showMessage(f"Updated: {name}")
15
16    def delete_item(self): 1 usage
17
18        if self.selected_id is None:
19            QMessageBox.information(self, title: "No Selection", text: "Select an item from the table first.")
20            return

```

```

        answer = QMessageBox.question(
            self,
            title: "Confirm Delete",
            text: "Are you sure you want to delete this item?",
            QMessageBox.Yes | QMessageBox.No,
        )
21
22        if answer != QMessageBox.Yes:
23            return
24
25        self.items = [item for item in self.items if item["id"] != self.selected_id]
26        inventory_data.save_items(self.items)
27
28        self.clear_form()
29        self.refresh_table()
30        self.statusbar.showMessage("Item deleted.")

```

```

31    def clear_form(self): 3 usages
32
33        self.selected_id = None
34        self.nameInput.clear()
35        self.quantityInput.setValue(0)
36        self.priceInput.setValue(0.0)
37        self.inventoryTable.clearSelection()
38
39    def on_row_selected(self): 1 usage
40
41        selected_rows = self.inventoryTable.selectionModel().selectedRows()
42        if not selected_rows:
43            return
44
45        row = selected_rows[0].row()
46        item_id = int(self.inventoryTable.item(row, 0).text())

```

```

    for item in self.items:
        if item["id"] == item_id:
            self.selected_id = item_id
            self.nameInput.setText(item["name"])
            self.quantityInput.setValue(item["quantity"])
            self.priceInput.setValue(item["price"])
            self.statusbar.showMessage(f"Editing item #{item_id}")
            break

```

DATA CODE:

```

import json
from pathlib import Path

DATA_FILE = Path(__file__).parent / "data" / "inventory.txt"

def load_items(): 1 usage

    if not DATA_FILE.exists():
        return []

    with open(DATA_FILE, "r", encoding="utf-8") as file:
        return json.load(file)

```

```

18 def save_items(items): 3 usages
19
20     DATA_FILE.parent.mkdir(parents=True, exist_ok=True)
21
22     with open(DATA_FILE, "w", encoding="utf-8") as file:
23         json.dump(items, file, indent=4)
24
25
26 def get_next_id(items): 1 usage
27
28     if not items:
29         return 1
30     return max(item["id"] for item in items) + 1

```

JSON CODE:


```
{
  {
    "id": 1,
    "name": "Notebook",
    "quantity": 50,
    "price": 2.99
  },
  {
    "id": 2,
    "name": "Pen",
    "quantity": 120,
    "price": 1.0
  },
  {
    "id": 3,
    "name": "USB Cable",
    "quantity": 40,
    "price": 8.5
  },
}
```

```
{
  "id": 4,
  "name": "Tissue",
  "quantity": 100,
  "price": 3.0
},
{
  "id": 5,
  "name": "Juice",
  "quantity": 200,
  "price": 50.0
}
```

MAIN WINDOW.UI:

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <!-- version="4.0" -->
3  <class>MainWindow</class>
4  <widget class="QMainWindow" name="MainWindow">
5  <property name="geometry">
6  <rect>
7  <x>0</x>
8  <y>0</y>
9  <width>800</width>
10 <height>500</height>
11 </rect>
12 </property>
13 <property name="windowTitle">
14 <string>Inventory Management System</string>
15 </property>
16 <widget class="QWidget" name="centralwidget">
17 <layout class="QVBoxLayout" name="verticalLayout">
18 <item>
19 <widget class="QLabel" name="titleLabel">
20 <property name="text">
21 <string>Simple Inventory Manager</string>
22 </property>
23 <property name="alignment">
24 <set>Qt::AlignCenter</set>
25 </property>
26 <property name="styleSheet">
27 <string>font-size: 18px; font-weight: bold; padding: 8px;</string>
28 </property>
29 </widget>
30 </item>
31 <item>
32 <layout class="QHBoxLayout" name="contentLayout">
33 <item>
34 <widget class="QGroupBox" name="formGroupBox">
35 <property name="title">
36 <string>Add / Edit Item</string>
37 </property>
38 <layout class="QFormLayout" name="formLayout">
39 <item row="0" column="0">
40 <widget class="QLabel" name="nameLabel">
41 <property name="text">
42 <string>Product Name:</string>
43 </property>
44 </widget>
45 </item>
46 <item row="0" column="1">
47 <widget class="QLineEdit" name="nameInput">
48 <property name="placeholderText">
49 <string>e.g. Notebook</string>
50 </property>
51 </widget>
52 </item>
```

```

52         </item>
53         <item row="1" column="0">
54             <widget class="QLabel" name="quantityLabel">
55                 <property name="text">
56                     <string>Quantity:</string>
57                 </property>
58             </widget>
59         </item>
60         <item row="1" column="1">
61             <widget class="QSpinBox" name="quantityInput">
62                 <property name="minimum">
63                     <number>0</number>
64                 </property>
65                 <property name="maximum">
66                     <number>99999</number>
67                 </property>
68             </widget>
69         </item>
70     <item row="2" column="0">
71         <widget class="QLabel" name="priceLabel">
72             <property name="text">
73                 <string>Price (<string>):</string>
74             </property>
75         </widget>
76     </item>
77     <item row="2" column="1">
78         <widget class="QDoubleSpinBox" name="priceInput">
79             <property name="decimals">
80                 <number>2</number>
81             </property>
82             <property name="maximum">
83                 <double>999999.99000000000005239</double>
84             </property>
85         </widget>
86     </item>
87     <item row="3" column="0" colspan="2">
88         <layout class="QHBoxLayout" name="buttonLayout">
89             <item>
90                 <widget class="QPushButton" name="addButton">
91                     <property name="text">
92                         <string>Add Item</string>
93                     </property>
94                 </widget>
95             </item>
96             <item>
97                 <widget class="QPushButton" name="updateButton">
98                     <property name="text">
99                         <string>Update</string>
100                 </property>
101             </widget>
102             </item>
103             <item>
104                 <widget class="QPushButton" name="deleteButton">

```

```

105         <property name="text">
106             <string>Delete</string>
107         </property>
108     </widget>
109 </item>
110 <item>
111     <widget class="QPushButton" name="clearButton">
112         <property name="text">
113             <string>Clear Form</string>
114         </property>
115     </widget>
116 </item>
117 </layout>
118 </item>
119 </layout>
120 </widget>
121 </item>
122 <item>
123     <widget class="QGroupBox" name="tableGroupBox">
124         <property name="title">
125             <string>Inventory List</string>
126         </property>
127         <layout class="QVBoxLayout" name="tableLayout">
128             <item>
129                 <widget class="QLineEdit" name="searchInput">
130                     <property name="placeholderText">
131                         <string>Search by product name...</string>
132                     </property>
133                 </widget>
134             </item>
135             <item>
136                 <widget class="QTableWidget" name="inventoryTable">
137                     <property name="editTriggers">
138                         <set>QAbstractItemView::NoEditTriggers</set>
139                     </property>
140                     <property name="selectionBehavior">

```

```
141         <enum>QAbstractItemView::SelectRows</enum>
142     </property>
143     <property name="selectionMode">
144         <enum>QAbstractItemView::SingleSelection</enum>
145     </property>
146     <column>
147         <property name="text">
148             <string>ID</string>
149         </property>
150     </column>
151     <column>
152         <property name="text">
153             <string>Product Name</string>
154         </property>
155     </column>
156     <column>
157         <property name="text">
```

```
158             <string>Quantity</string>
159         </property>
160     </column>
161     <column>
162         <property name="text">
163             <string>Price ($)</string>
164         </property>
165     </column>
166 </widget>
167 </item>
168 </layout>
169 </widget>
170 </item>
171 </layout>
172 </item>
173 </layout>
174 </widget>
```

```
174 </widget>
175 <widget class="QStatusBar" name="statusbar"/>
176 </widget>
177 <resources/>
178 <connections/>
179 </ui>
180
```

CONCLUSION:

This project successfully demonstrates the implementation of an Inventory Management System using Python, JSON, and Qt Designer. It provides practical experience in GUI development, file handling, and data management. The system efficiently performs all required operations and serves as a strong foundation for developing more advanced software applications in the future.